

Web Development

Labo 18

Jari Rhellam

1. DOM tree elementen opvragen

Methodes :

```
document.getElementById("id")  
document.getElementsByClassName("class")  
document.getElementsByTagName("p")
```

Wat doen ze?

- getElementById → 1 element
- getElementsByClassName → lijst (HTMLCollection)
- getElementsByTagName → lijst

Of

```
document.querySelector("selector")  
document.querySelectorAll("selector")
```

Gebruikt CSS selectors

Bijvoorbeeld:

```
document.querySelector("#playfield") // id  
document.querySelector(".box")      // class  
document.querySelector("p")          // tag  
  
document.querySelectorAll(".important>img")
```

Opdracht 1 events

```
const setup = () => {  
  const paragrafen = document.querySelectorAll(".text");  
  
  paragrafen.forEach(p => {  
    p.addEventListener("click", (event) => {  
      event.target.style.color = "red";  
    });  
  });  
};
```

```
};  
  
window.addEventListener("load", setup);
```

Opdracht 2 event bubbling

Event bubbling betekent dat wanneer je op een element klikt, het event automatisch omhoog bubbelt door de DOM-tree naar alle ancestor-elementen. Elk ancestor met een click-listener krijgt ook de kans om te reageren.

`event.target` = het element waar het event begon (blijft altijd hetzelfde).

`event.currentTarget` = het element wiens listener nu uitgevoerd wordt (verandert per niveau). Met `event.stopPropagation()` stop je het bubbling. Met `event.preventDefault()` stop je het standaard browsergedrag (bv. een link volgen).

Observatie demo: klik op de a-link → 3 logs in console: `target=A/currentTarget=A`, `target=A/currentTarget=LI`, `target=A/currentTarget=UL`. `Event.target` blijft A, `currentTarget` verandert per niveau.

```
const setup = () => {  
  document.querySelector("ul").addEventListener("click", () => {  
    console.log("UL geklikt");  
  });  
  
  document.querySelectorAll("li").forEach(li => {  
    li.addEventListener("click", () => {  
      console.log("LI geklikt");  
    });  
  });  
  
  document.querySelectorAll("a").forEach(a => {  
    a.addEventListener("click", (event) => {  
      console.log("A geklikt");  
      event.preventDefault();  
    });  
  });  
};  
  
window.addEventListener("load", setup);
```

Opdracht This keyword

In een gewone functie (niet-arrow) verwijst `this` in een event listener naar het element wiens listener wordt uitgevoerd. In een arrow function werkt `this` anders: het verwijst naar de omliggende scope (meestal `window`), waardoor `this.innerHTML` undefined geeft.

Oplossing: gebruik altijd `event.currentTarget` (of `event.target`) in plaats van `this`. Dat werkt correct in zowel gewone functies als arrow functions.

```
const setup = () => {
  const p = document.getElementById("txtParagraaf");

  p.addEventListener("click", klik);
};

function klik() {
  this.style.color = "blue";
}

window.addEventListener("load", setup);
```

Opdracht DOM

De DOM-tree bevat niet alleen element nodes, maar ook text nodes (de tekst tussen tags), comment nodes, etc. Text nodes bevatten ook whitespace en newlines uit de HTML-broncode.

Navigatie-properties: `n.parentNode`, `n.childNodes` (alle kinderen incl. text nodes), `n.children` (enkel element nodes), `n.firstChild/lastChild`, `n.firstElementChild/lastElementChild`. Type-properties: `n.nodeType` (1=element, 3=text), `n.nodeName`, `n.nodeValue` (tekst van text node).

Observatie debugger: `childNodes[0]` van de `p` is een TEXT NODE met `nodeValue` `"\n Hello World!\n "` (inclusief alle whitespace en newlines). `childNodes[1]` is de SPAN ELEMENT NODE. `childNodes[2]` is een TEXT NODE met `"tekst"`.

```
const setup = () => {
  const p = document.getElementById("abc");

  console.log(p.childNodes); // toont alle nodes
  console.log(p.children);  // enkel elementen

  // voorbeeld: tekst aanpassen
```

```
p.firstChild.nodeValue = "Aangepaste tekst ";  
};  
  
window.addEventListener("load", setup);
```

Opdracht Nodes 1

```
const setup = () => {  
  let paragraaf = document.querySelectorAll("p");  
  paragraaf.forEach(p => {p.textContent = "Goed gedaan!"})  
}  
addEventListener("load", setup)
```

Opdracht Nodes 2

```
const setup = () => {  
  let list = document.querySelectorAll("li");  
  
  for (let i = 0; i < list.length; i++) {  
    list[i].classList.add("listitem");  
  }  
  
  let img = document.createElement("img");  
  img.setAttribute("src", "sunset.jpg")  
  img.setAttribute("alt", "sunset");  
  document.body.appendChild(img);  
  
}  
window.addEventListener("load", setup);
```

Opdracht Nodes 3

```
const setup = () => {  
  const knop = document.querySelector("#btn");  
  const div = document.querySelector("#myDIV");  
  
  knop.addEventListener("click", () => {  
    const p = document.createElement("p");
```

```
p.textContent = "Nieuw p element";
div.appendChild(p);
});
}
window.addEventListener("load", setup);
```

Opdracht Colorpicker

```
const setup = () => {
  const sliders = document.getElementsByClassName("slider");

  for (let i = 0; i < sliders.length; i++) {
    sliders[i].addEventListener("input", update);
    sliders[i].addEventListener("change", update);
  }

  document.getElementById("btnSave").addEventListener("click", saveColor);

  update();
};

const update = () => {
  const valueRed = document.getElementById("sldrood").value;
  const valueGreen = document.getElementById("sldgroen").value;
  const valueBlue = document.getElementById("sldblauw").value;

  document.getElementById("rood").innerHTML = valueRed;
  document.getElementById("groen").innerHTML = valueGreen;
  document.getElementById("blauw").innerHTML = valueBlue;

  const swatch = document.getElementById("swatch");
  swatch.style.backgroundColor = `rgb(${valueRed}, ${valueGreen}, ${valueBlue})`;
};

const saveColor = () => {
  const valueRed = document.getElementById("sldrood").value;
  const valueGreen = document.getElementById("sldgroen").value;
  const valueBlue = document.getElementById("sldblauw").value;

  const divColors = document.getElementById("divColor");

  const obj = document.createElement("div");
  obj.style.backgroundColor = `rgb(${valueRed}, ${valueGreen}, ${valueBlue})`;
  obj.style.position = "relative";
```

```

obj.style.width = "70px";
obj.style.height = "70px";
obj.style.margin = "5px";
obj.style.display = "inline-block";

// Sla RGB op als data-attributen zodat restoreColor ze kan ophalen
obj.setAttribute("data-r", valueRed);
obj.setAttribute("data-g", valueGreen);
obj.setAttribute("data-b", valueBlue);

obj.addEventListener("click", restoreColor);

const btnDelete = document.createElement("button");
btnDelete.textContent = "x";
btnDelete.style.position = "absolute";
btnDelete.style.top = "2px";
btnDelete.style.right = "2px";

btnDelete.addEventListener("click", (event) => {
  event.stopPropagation(); // anders triggert de klik ook restoreColor!
  event.currentTarget.parentElement.remove();
});

divColors.appendChild(obj);
obj.appendChild(btnDelete);
};

const restoreColor = (event) => {
  const swatch = event.currentTarget;

  document.getElementById("sldrood").value = swatch.getAttribute("data-r");
  document.getElementById("sldgroen").value = swatch.getAttribute("data-g");
  document.getElementById("sldblauw").value = swatch.getAttribute("data-b");

  update();
};

window.addEventListener("load", setup);

```

Html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">

```

```

<link rel="stylesheet" href="colorPicker.css">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Color Picker</title>
</head>
<body>
<div class="colorPicker">
  <div class="components">

    <div class="component">
      <span>0 <label for="sldrood"></label><input id="sldrood" type="range"
class="slider" value="128" min="0" max="255"> 255</span>
      <span>Rood</span>
      <span id="rood">128</span>
    </div>

    <div class="component">
      <span>0 <label for="sldgroen"></label><input id="sldgroen" type="range"
class="slider" value="128" min="0" max="255"> 255</span>
      <span>Groen</span>
      <span id="groen">128</span>
    </div>

    <div class="component">
      <span>0 <label for="sldblauw"></label><input id="sldblauw" type="range"
class="slider" value="128" min="0" max="255"> 255</span>
      <span>Blauw</span>
      <span id="blauw">128</span>
    </div>

  </div>

  <div class="swatch" id="swatch"></div>
  <input type="button" id="btnSave" value="save">
</div>
<div id="divColor"></div>

<script src="scripts/colorPicker.js"></script>
</body>
</html>

```